1. Write a program to perform the following

An empty list

A list with one element

A list with all identical elements

A list with negative numbers

Test Cases:

1. Input: []

- **Expected Output: []**

1. Input: [1]

- **Expected Output: [1]**

2. Input: [7, 7, 7, 7]

- **Expected Output: [7, 7, 7, 7]**

3. Input: [-5, -1, -3, -2, -4]

- **Expected Output: [-5, -4, -3, -2, -1]**

Aim

To sort different types of lists using a brute force approach.

Algorithm

1. Take the input list.
2. Use simple sorting (nested loops).
3. Compare each element and swap if needed.
4. Return sorted list.

Python Code

File Edit Format Run Options Windows Help

```
def brute_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(i+1, n):
            if arr[i] > arr[j]:
                arr[i], arr[j] = arr[j], arr[i]
    return arr

# Test Cases
print(brute_sort([]))
print(brute_sort([1]))
print(brute_sort([7,7,7,7]))
print(brute_sort([-5,-1,-3,-2,-4]))
```

Input

[]
[1]
[7, 7, 7, 7]
[-5, -1, -3, -2, -4]

Output

```
>>> ================================ RESTART ================================
>>>
[]
[1]
[7, 7, 7, 7]
[-5, -4, -3, -2, -1]
>>>
```

Result

The program sorts lists using a brute force comparison method.
Time complexity is $O(n^2)$, suitable only for small inputs.

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain

why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted Array:

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Aim

To sort an array using Selection Sort and understand its working.

Algorithm

1. Start from first element.
2. Find smallest element in remaining array.
3. Swap with current position.
4. Repeat for all elements.

Python Code

File Edit Format Run Options Windows Help

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# Test Cases
print(selection_sort([5,2,9,1,5,6]))
print(selection_sort([10,8,6,4,2]))
print(selection_sort([1,2,3,4,5]))
```

Input

[5, 2, 9, 1, 5, 6]
[10, 8, 6, 4, 2]
[1, 2, 3, 4, 5]

Output

```
>>> ================================ RESTART ================================
>>>
[1, 2, 5, 5, 6, 9]
[2, 4, 6, 8, 10]
[1, 2, 3, 4, 5]
>>>
```

Result

Selection sort repeatedly selects the minimum element and places it correctly.
Time complexity is $O(n^2)$, making it inefficient for large datasets.

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

Test Cases:

- **Test your optimized function with the following lists:**

1. **Input: [64, 25, 12, 22, 11]**
   - **Expected Output: [11, 12, 22, 25, 64]**
2. **Input: [29, 10, 14, 37, 13]**
   - **Expected Output: [10, 13, 14, 29, 37]**
3. **Input: [3, 5, 2, 1, 4]**
   - **Expected Output: [1, 2, 3, 4, 5]**
4. **Input: [1, 2, 3, 4, 5] (Already sorted list)**
   - **Expected Output: [1, 2, 3, 4, 5]**
5. **Input: [5, 4, 3, 2, 1] (Reverse sorted list)**
   - **Expected Output: [1, 2, 3, 4, 5]**

Aim

To modify bubble sort so it stops early if the list is already sorted.

Algorithm

1. Traverse the list.
2. Swap adjacent elements if needed.
3. Use a flag to check if any swap happened.
4. If no swap → stop early.

Python Code

File Edit Format Run Options Windows Help

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

# Inputs for IDLE
print(bubble_sort([64, 25, 12, 22, 11]))
print(bubble_sort([29, 10, 14, 37, 13]))
print(bubble_sort([3, 5, 2, 1, 4]))
print(bubble_sort([1, 2, 3, 4, 5]))
print(bubble_sort([5, 4, 3, 2, 1]))
```

Input

[64, 25, 12, 22, 11]
[29, 10, 14, 37, 13]
[3, 5, 2, 1, 4]
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]

Output

```
>>> ================================ RESTART ================================
>>>
[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
>>>
>>>
```

Result

The optimized bubble sort stops early when the list is already sorted.
Best case time complexity improves to O(n).

4.Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

Examples:

1. Array with Duplicates:

- **Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]**
- **Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]**

2. All Identical Elements:

- **Input: [5, 5, 5, 5, 5]**
- **Output: [5, 5, 5, 5, 5]**

3. Mixed Duplicates:

- **Input: [2, 3, 1, 3, 2, 1, 1, 3]**
- **Output: [1, 1, 1, 2, 2, 3, 3, 3]**

Aim

To sort an array using insertion sort and handle duplicate elements correctly.

Algorithm

1. Start from second element.
2. Compare with previous elements.
3. Shift larger elements right.
4. Insert element in correct position.
5. Maintain order of duplicates (stable).

Python Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1

        arr[j+1] = key

    return arr

# Inputs for IDLE
print(insertion_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
print(insertion_sort([5, 5, 5, 5, 5]))
print(insertion_sort([2, 3, 1, 3, 2, 1, 1, 3]))
```

Input

[3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
[5, 5, 5, 5, 5]
[2, 3, 1, 3, 2, 1, 1, 3]

Output

```
>>> ================================ RESTART ================================
>>>
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]
>>>
>>>
```

Result

Insertion sort correctly handles duplicate values and maintains their order.
It is a stable sorting algorithm with $O(n^2)$ time complexity.

5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return the kth positive integer that is missing from this array.

Example 1:

Input: arr = [2,3,4,7,11], k = 5

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.

Example 2:

Input: arr = [1,2,3,4], k = 2

Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.

Aim

To find the kth missing positive integer from a sorted array.

Algorithm

1. Initialize current number = 1 and index = 0.
2. Traverse numbers while counting missing elements.
3. If current number not in array → increment missing count.
4. When count = k → return current number.

Python Code

File Edit Format Run Options Windows Help

```
def findKthPositive(arr, k):
    i, num = 0, 1

    while True:
        if i < len(arr) and arr[i] == num:
            i += 1
        else:
            k -= 1
            if k == 0:
                return num
        num += 1

# Inputs for IDLE
print(findKthPositive([2,3,4,7,11], 5))
print(findKthPositive([1,2,3,4], 2))
```

Input

arr = [2,3,4,7,11], k = 5
arr = [1,2,3,4], k = 2

Output

```
>>> ================================ RESTART ================================
>>>
9
6
>>>
```

Result

The program finds missing numbers by sequential checking.
Time complexity is O(n + k).

6. **A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in O(log n) time.**

Example 1:

Input: nums = [1,2,3,1]

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: nums = [1,2,1,3,5,6,4]

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Aim

To find a peak element index in an array using O(log n) time.

Algorithm

1. Use binary search.
2. Find middle element.
3. If mid < next → move right.
4. Else → move left.
5. Return index where peak found.

Python Code

File Edit Format Run Options Windows Help

```
def findPeakElement(nums):
    low, high = 0, len(nums) - 1

    while low < high:
        mid = (low + high) // 2

        if nums[mid] < nums[mid + 1]:
            low = mid + 1
        else:
            high = mid

    return low

# Inputs for IDLE
print(findPeakElement([1,2,3,1]))
print(findPeakElement([1,2,1,3,5,6,4]))
```

Input

[1,2,3,1]
[1,2,1,3,5,6,4]

Output

```
>>> ================================ RESTART ================================
>>>
2
5
>>>
```

Result

Binary search efficiently finds a peak element.
Time complexity is O(log n).

7. Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

Example 2:

Input: haystack = "leetcode", needle = "leeto"

Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

Aim

To find the index of the first occurrence of a substring (needle) in a string (haystack).

Algorithm

1. Traverse the main string.
2. Compare substring with each position.
3. If match found → return index.
4. If no match → return -1.

Python Code

File Edit Format Run Options Windows Help

```
def find_index(haystack, needle):
    n, m = len(haystack), len(needle)

    for i in range(n - m + 1):
        if haystack[i:i+m] == needle:
            return i
    return -1

# Test Cases
print(find_index("sadbutsad", "sad"))
print(find_index("leetcode", "leeto"))
```

Input

haystack = "sadbutsad", needle = "sad"
haystack = "leetcode", needle = "leeto"

Output

```
>>> ================================ RESTART ================================
>>>
0
-1
>>>
```

Result

The program finds the first occurrence using simple comparison. Time complexity is O(n × m).

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

Example 2:

Input: words = ["leetcode","et","code"]

Output: ["et","code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue","green","bu"]

Output: []

Explanation: No string of words is substring of another string.

Aim

To find all strings in a list that are substrings of another string.

Algorithm

1. Compare each word with every other word.
2. If word is found inside another → add to result.
3. Avoid duplicates.
4. Return result list.

Python Code

File Edit Format Run Options Windows Help

```
def string_matching(words):
    result = []

    for i in range(len(words)):
        for j in range(len(words)):
            if i != j and words[i] in words[j]:
                result.append(words[i])
                break

    return result

# Test Cases
print(string_matching(["mass","as","hero","superhero"]))
print(string_matching(["leetcode","et","code"]))
print(string_matching(["blue","green","bu"]))
```

Input

["mass","as","hero","superhero"]
["leetcode","et","code"]
["blue","green","bu"]

Output

```
>>> ================================ RESTART ================================
>>>
['as', 'hero']
['et', 'code']
[]
>>>
>>>
```

Result

The program identifies substrings by comparing each word with others.
Time complexity is $O(n^2 \times k)$.

9. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

The two points with the minimum distance between them.

The minimum distance itself.

Closest pair: (1, 2) - (3, 1) Minimum distance: 1.4142135623730951

Aim

To find the closest pair of points in a 2D plane using brute force.

Algorithm

1. Take all pairs of points.
2. Compute distance between each pair.
3. Keep track of minimum distance.
4. Return closest pair and distance.

Python Code

```
import math

def distance(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def closest_pair(points):
    min_dist = float('inf')
    pair = None

    for i in range(len(points)):
        for j in range(i+1, len(points)):
            d = distance(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])

    return pair, min_dist

# Input
points = [(1,2), (4,5), (7,8), (3,1)]

# Output
pair, dist = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", dist)
```

Input

[(1, 2), (4, 5), (7, 8), (3, 1)]

Output

```
>>> ================================ RESTART ================================
>>>
('Closest pair:', (1, 2), '-', (3, 1))
('Minimum distance:', 2.23606797749979)
>>>
>>>
```

Result

The program checks all pairs to find minimum distance. Time complexity is $O(n^2)$.

10. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

Aim

To find closest pair of points and compute convex hull using brute force approach.

Algorithm (Closest Pair)

1. Use distance formula.
2. Compare all pairs.
3. Track minimum distance.

Algorithm (Convex Hull - Brute Force)

1. For every pair of points, form a line.
2. Check if all other points lie on one side.
3. If yes, the line is part of convex hull.
4. Collect such points.

Python Code

```
import math

points = [(10,0),(11,5),(5,3),(9,3.5),(15,3),(12.5,7),(6,6.5),(7.5,4.5)]

def distance(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

# Closest Pair
def closest_pair(points):
    min_dist = float('inf')
    pair = None
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            d = distance(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist
```

```
# Convex Hull (Brute Force)
def convex_hull(points):
    hull = set()
    n = len(points)

    for i in range(n):
        for j in range(i+1, n):
            left = right = 0
            x1,y1 = points[i]
            x2,y2 = points[j]

            for k in range(n):
                if k == i or k == j:
                    continue
                x,y = points[k]
                val = (y2-y1)*(x-x2) - (x2-x1)*(y-y2)
                if val > 0:
                    left += 1
                elif val < 0:
                    right += 1

            if left == 0 or right == 0:
                hull.add(points[i])
                hull.add(points[j])

    return list(hull)

# Execution
pair, dist = closest_pair(points)
hull = convex_hull(points)

print("Closest Pair:", pair, "Distance:", dist)
print("Convex Hull:", hull)
```

Input

P1 (10,0), P2 (11,5), P3 (5,3), P4 (9,3.5),
P5 (15,3), P6 (12.5,7), P7 (6,6.5), P8 (7.5,4.5)

Output

```
>>> ================================ RESTART ================================
>>>
('Closest Pair:', ((9, 3.5), (7.5, 4.5)), 'Distance:', 1.8027756377319946)
('Convex Hull:', [(10, 0), (12.5, 7), (6, 6.5), (15, 3), (5, 3)])
>>>
>>>
>>>
```

Result

The brute force method checks all point pairs and lines.
Time complexity is $O(n^2)$ for closest pair and $O(n^3)$ for convex hull.

11. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output:

The list of points that form the convex hull in counter-clockwise order.

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

Aim

To find the convex hull of a set of 2D points using brute force.

Algorithm

1. Take every pair of points and form a line.
2. Check position of all other points relative to the line.
3. If all points lie on one side → edge of convex hull.
4. Collect such points and arrange in counter-clockwise order.

Python Code

```
def convex_hull(points):
    hull = set()
    n = len(points)

    for i in range(n):
        for j in range(i+1, n):
            left = right = 0
            x1,y1 = points[i]
            x2,y2 = points[j]

            for k in range(n):
                if k == i or k == j:
                    continue
                x,y = points[k]
                val = (y2-y1)*(x-x2) - (x2-x1)*(y-y2)
                if val > 0:
                    left += 1
                elif val < 0:
                    right += 1

            if left == 0 or right == 0:
                hull.add(points[i])
                hull.add(points[j])

    return sorted(list(hull))

# Input
points = [(1,1), (4,6), (8,1), (0,0), (3,3)]

# Output
print("Convex Hull:", convex_hull(points))
```

Input

[(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output

```
>>> ================================ RESTART ================================
>>>
('Convex Hull:', [(0, 0), (4, 6), (8, 1)])
>>>
```

Result

The program finds boundary points by checking all possible lines.
Time complexity is $O(n^3)$, suitable for small datasets.

12. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function distance(city1, city2) to calculate the distance between two cities (e.g., Euclidean distance).

2. Implement a function tsp(cities) that takes a list of cities as input and performs the following:

• Generate all possible permutations of the cities (excluding the starting city) using itertools.permutations.

• For each permutation (representing a potential route):

• Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.

• Keep track of the shortest distance encountered and the corresponding path.

• Return the minimum distance and the shortest path (including the starting city at the beginning and end).

3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])

More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

Aim

To find the shortest possible route that visits all cities exactly once and returns to the starting city using exhaustive search.

Algorithm

1. Define distance between two cities using Euclidean formula.
2. Fix the first city as starting point.
3. Generate all permutations of remaining cities.
4. For each permutation:
    a. Form a complete path (start → cities → start).
    b. Compute total distance.
5. Track the minimum distance and corresponding path.
6. Return shortest path and distance.

Python Code

```
import itertools
import math

def distance(c1, c2):
    return math.sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)

def tsp(cities):
    start = cities[0]
    min_dist = float('inf')
    best_path = []

    for perm in itertools.permutations(cities[1:]):
        path = [start] + list(perm) + [start]
        total = 0

        for i in range(len(path)-1):
            total += distance(path[i], path[i+1])

        if total < min_dist:
            min_dist = total
            best_path = path

    return best_path, min_dist

# Test Case 1
cities1 = [(1,2), (4,5), (7,1), (3,6)]
path1, dist1 = tsp(cities1)
print("Test Case 1:")
print("Shortest Distance:", dist1)
print("Shortest Path:", path1)

# Test Case 2
cities2 = [(2,4), (8,1), (1,7), (6,3), (5,9)]
path2, dist2 = tsp(cities2)
print("\nTest Case 2:")
print("Shortest Distance:", dist2)
print("Shortest Path:", path2)
```

Input

Test Case 1: [(1, 2), (4, 5), (7, 1), (3, 6)]
Test Case 2: [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]

Output

```
>>> ================================ RESTART ================================
>>>
Test Case 1:
('Shortest Distance:', 16.969112047670894)
('Shortest Path:', [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)])

Test Case 2:
('Shortest Distance:', 23.12995011084934)
('Shortest Path:', [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)])
```

Result

The program generates all possible routes and selects the shortest one.
Time complexity is O(n!), making it suitable only for small datasets.

13. **You are given a cost matrix where each element cost[i][j] represents the cost of assigning worker i to task j. Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function total_cost(assignment, cost_matrix) that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function assignment_problem(cost_matrix) that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).**

    Test Cases:

    Input

    Simple Case: Cost Matrix:

    [[3, 10, 7],

    [8, 5, 12],

    [4, 6, 9]]

    More Complex Case: Cost Matrix:

    [[15, 9, 4],

    [8, 7, 18],

[6, 12, 11]]

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

Aim

To find the optimal assignment of workers to tasks with minimum total cost using exhaustive search.

Algorithm

1. Generate all permutations of task assignments.
2. For each permutation:
    a. Calculate total cost.
3. Track minimum cost and corresponding assignment.
4. Return optimal assignment and cost.

Python Code

```
import itertools

def total_cost(assignment, cost_matrix):
    cost = 0
    for i in range(len(assignment)):
        cost += cost_matrix[i][assignment[i]]
    return cost

def assignment_problem(cost_matrix):
    n = len(cost_matrix)
    perms = itertools.permutations(range(n))

    min_cost = float('inf')
    best_assign = None

    for p in perms:
        c = total_cost(p, cost_matrix)
        if c < min_cost:
            min_cost = c
            best_assign = p

    return best_assign, min_cost

# Test Case 1
cost1 = [[3,10,7],[8,5,12],[4,6,9]]
assign1, cost_val1 = assignment_problem(cost1)

print("Test Case 1:")
print("Optimal Assignment:", assign1)
print("Total Cost:", cost_val1)

# Test Case 2
cost2 = [[15,9,4],[8,7,18],[6,12,11]]
assign2, cost_val2 = assignment_problem(cost2)

print("\nTest Case 2:")
print("Optimal Assignment:", assign2)
print("Total Cost:", cost_val2)
```

Input

Test Case 1:
[[3,10,7],[8,5,12],[4,6,9]]

Test Case 2:
[[15,9,4],[8,7,18],[6,12,11]]

Output

```
>>> ================================ RESTART ================================
>>>
Test Case 1:
('Optimal Assignment:', (2, 1, 0))
('Total Cost:', 16)

Test Case 2:
('Optimal Assignment:', (2, 1, 0))
('Total Cost:', 17)
>>>
```

Result

The program checks all assignments to find the minimum cost solution. Time complexity is O(n!).

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

Define a function total_value(items, values) that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.

Define a function is_feasible(items, weights, capacity) that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

Simple Case:

Items: 3 (represented by indices 0, 1, 2)

Weights: [2, 3, 1]

Values: [4, 5, 3]

Capacity: 4

More Complex Case:

Items: 4 (represented by indices 0, 1, 2, 3)

Weights: [1, 2, 3, 4]

Values: [2, 4, 6, 3]

Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10

Aim

To select items that maximize value without exceeding capacity using exhaustive search.

Algorithm

1. Generate all subsets of items.
2. For each subset:
    a. Check feasibility (weight ≤ capacity).
    b. Calculate total value.
3. Track maximum value and best subset.
4. Return optimal selection.

Python Code

```
import itertools

def total_value(items, values):
    return sum(values[i] for i in items)

def is_feasible(items, weights, capacity):
    return sum(weights[i] for i in items) <= capacity

def knapsack(weights, values, capacity):
    n = len(weights)
    best_value = 0
    best_set = []

    for r in range(n+1):
        for subset in itertools.combinations(range(n), r):
            if is_feasible(subset, weights, capacity):
                val = total_value(subset, values)
                if val > best_value:
                    best_value = val
                    best_set = subset

    return list(best_set), best_value

# Test Case 1
weights1 = [2,3,1]
values1 = [4,5,3]
cap1 = 4

res1 = knapsack(weights1, values1, cap1)
print("Test Case 1:")
print("Optimal Selection:", res1[0])
print("Total Value:", res1[1])

# Test Case 2
weights2 = [1,2,3,4]
values2 = [2,4,6,3]
cap2 = 6

res2 = knapsack(weights2, values2, cap2)
print("\nTest Case 2:")
print("Optimal Selection:", res2[0])
print("Total Value:", res2[1])
```

Input

Test Case 1:
Weights: [2,3,1], Values: [4,5,3], Capacity: 4

Test Case 2:
Weights: [1,2,3,4], Values: [2,4,6,3], Capacity: 6

Output

```
>>> ================================ RESTART ================================
>>>
Test Case 1:
('Optimal Selection:', [1, 2])
('Total Value:', 8)

Test Case 2:
('Optimal Selection:', [0, 1, 2])
('Total Value:', 12)
```

Result

The program evaluates all subsets to maximize value within capacity.
Time complexity is $O(2^n)$.